Experiment 9

Aim

To design and train a **Convolutional Neural Network (CNN)** for image classification using the Fashion-MNIST dataset.

---

1. Dataset Source

Source:
https://github.com/zalandoresearch/fashion-mnist

---

2. Dataset Description

Total Images: 70,000

- Training: 60,000
- Testing: 10,000

Image Size: 28 × 28 pixels (grayscale)

Classes (10 categories):

- T-shirt/top
- Trouser
- Pullover
- Dress
- Coat
- Sandal
- Shirt
- Sneaker
- Bag
- Ankle boot

Target Variable:

- Label (0–9 representing clothing category)

Description:
Fashion-MNIST is a dataset of clothing images used for benchmarking image classification models and is more complex than the traditional MNIST dataset.

---

3. Mathematical Formulation of the Algorithm

Convolution Operation

$$(I * K)(x, y) = \sum_m \sum_n I(x - m, y - n) \cdot K(m, n)$$

ReLU Activation

$$f(x) = \max(0, x)$$

Softmax Function

$$P(y_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

Loss Function (Categorical Crossentropy)

$$L = -\sum y_i \log(\hat{y}_i)$$

---

4. Algorithm Limitations

- Requires high computational power
- Needs large dataset for better accuracy
- Prone to overfitting
- Training time is high

---

5. Methodology / Workflow

1. Load dataset
2. Normalize pixel values
3. Reshape images for CNN
4. Build CNN model
5. Compile model
6. Train model
7. Evaluate performance
8. Predict results

---

6. Performance Analysis

- Metric: Accuracy
- CNN typically achieves **~90–93% accuracy** on Fashion-MNIST
- Better than traditional ANN due to spatial feature extraction

---

7. Hyperparameter Tuning

- Number of convolution layers
- Filter size (3×3, 5×5)
- Batch size
- Epochs
- Learning rate

Code

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
```

```
# Load dataset (auto-download)
(X_train, y_train), (X_test, y_test) =
keras.datasets.fashion_mnist.load_data()

# Normalize data
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape for CNN (add channel)
X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)

# Build CNN Model
model = keras.Sequential([
    keras.layers.Conv2D(32, (3,3), activation='relu',
input_shape=(28,28,1)),
    keras.layers.MaxPooling2D((2,2)),

    keras.layers.Conv2D(64, (3,3), activation='relu'),
    keras.layers.MaxPooling2D((2,2)),

    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])

# Compile model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train model
model.fit(X_train, y_train, epochs=5, batch_size=32)
```

```
# Evaluate
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)

# Prediction
predictions = model.predict(X_test)
print(np.argmax(predictions[0]))
```

Output

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 34s 18ms/step - accuracy: 0.8382 - loss: 0.4443
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 33s 18ms/step - accuracy: 0.8907 - loss: 0.2970
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 33s 18ms/step - accuracy: 0.9057 - loss: 0.2518
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 33s 18ms/step - accuracy: 0.9182 - loss: 0.2197
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 33s 18ms/step - accuracy: 0.9266 - loss: 0.1943
<keras.src.callbacks.history.History at 0x7eddab508b00>
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9079 - loss: 0.2523
Accuracy: 0.9078999757766724
```

```python
# Cell 8: Prediction
predictions = model.predict(X_test)

print("Predicted:", np.argmax(predictions[0]))
print("Actual:", y_test[0])
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step
Predicted: 9
Actual: 9
```

Conclusion

The CNN model successfully classifies fashion images with high accuracy.

CNN performs better than ANN for image data because it captures spatial features effectively.